Chapter 12: Python Database

Contents

Chapter 12: Python Database 1
SQL Online Tutorial 2
SQLite with Python Tutorials 2
SQLite and Python 2
Self-Contained 3
Zero-Configuration 3
Transactional (ACID) 3
Single Database File 3
CRUD 4
Tutorial 1: DB Browser for SQLite 5
Install SQLite DB Browser for SQLite 5
Create Database 5
Tutorial 2: Create a Database with Python 7
Explanation: How to Connect to an SQLite Database in Python 7
Tutorial 3: Create a Table 8
Explanation 9
Tutorial 4: Insert Records 11
Tutorial 5: Select Records 12
Tutorial 6: Update Records 12
Tutorial 7: Delete Records 13
Assignment Submission 13

Time required: 90 minutes

- Follow all directions carefully and accurately.
- Think of the directions as minimum requirements.

SQL Online Tutorial

- https://www.w3schools.com/sql/sql_intro.asp
- https://www.w3schools.com/sql/sql_syntax.asp
- https://www.w3schools.com/sql/sql_create_db.asp

SQLite with Python Tutorials

- SQLite Databases with Python - Full Course – FreeCodeCamp.org
- https://www.sqlitetutorial.net

SQLite and Python

SQLite is a self-contained, file-based SQL database. SQLite comes bundled with Python and can be used in any of your Python applications without having to install any additional software.

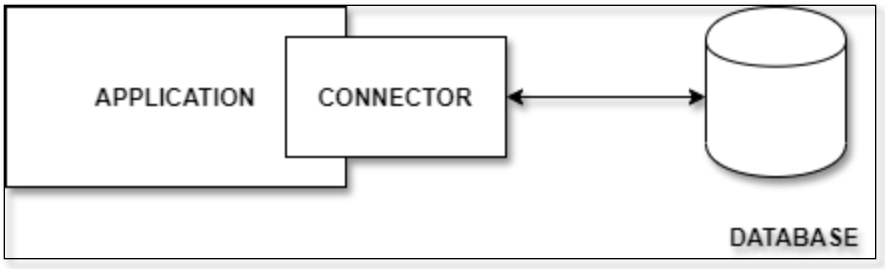
Databases offer numerous functionalities by which one can manage large amounts of information easily over the web and high-volume data input and output over a typical file such as a text file. SQL is a query language and is very popular in databases. Many websites use MySQL. SQLite is a "light" version that works over syntax very similar to SQL.

SQLite is a self-contained, high-reliability, embedded, full-featured, public-domain, SQL database engine. It is the most used database engine on the world wide web. Python has a library to access SQLite databases, called sqlite3, intended for working with SQLite.

SQLite has the following features.

1. Serverless
2. Self-Contained
3. Zero-Configuration
4. Transactional (ACID)
5. Single Database file

SQLite does not require a server to run. The SQLite database is joined with the application that accesses the database. SQLite database read and write directly from the database files stored on disk and applications interact with that SQLite database.



Self-Contained

SQLite does not need any external dependencies like an operating system or external library. This feature of SQLite helps especially in embedded devices like iPhones, Android phones, game consoles, handheld media players, etc.

Zero-Configuration

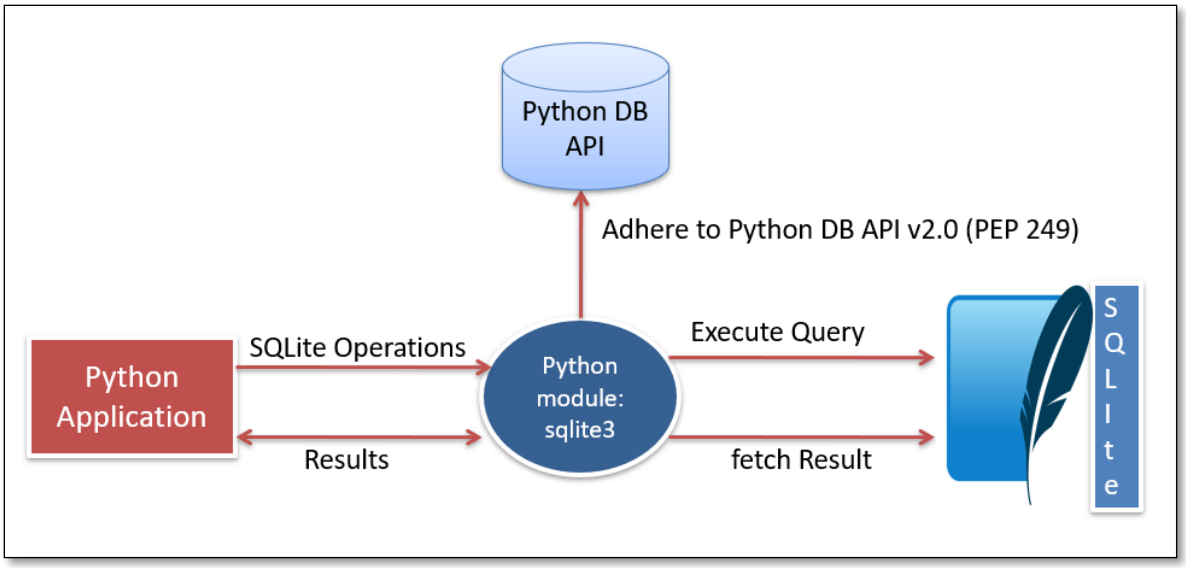
Zero-configuration means no setup or administration needed. Because of the serverless architecture, you don't need to "install" SQLite before using it. There is no server process that needs to be configured, started, and stopped.

Transactional (ACID)

Transactional means they are atomic, consistent, isolated, and durable (ACID). All transactions in SQLite are fully ACID-compliant. In other words, all changes within a transaction take place completely or not at all even when an unexpected situation like application crash, power failure, or operating system crash occurs.

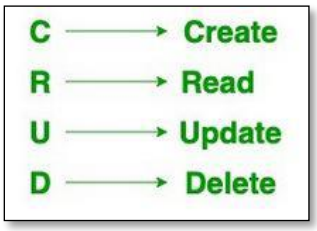
Single Database File

SQLite is a single database that means it allows a single database connection to access multiple database files simultaneously. These features bring many nice features like joining tables in different databases or copying data between databases in a single command. SQLite also uses dynamic types for tables. It means you can store any value in any column, regardless of the data type.



CRUD

A database provides four major operations, usually referred to as CRUD.



The corresponding SQL commands. Notice that SQL commands are typically in UPPER CASE. This is a convention; SQL does not require UPPER CASE. It makes the SQL commands easier to pick out of an SQL statement

CREATE: Create new tables and records

SELECT: Select all or selected fields and records

UPDATE: Modify existing tables and records

DELETE: Delete existing tables and records

Tutorial 1: DB Browser for SQLite

Install SQLite DB Browser for SQLite

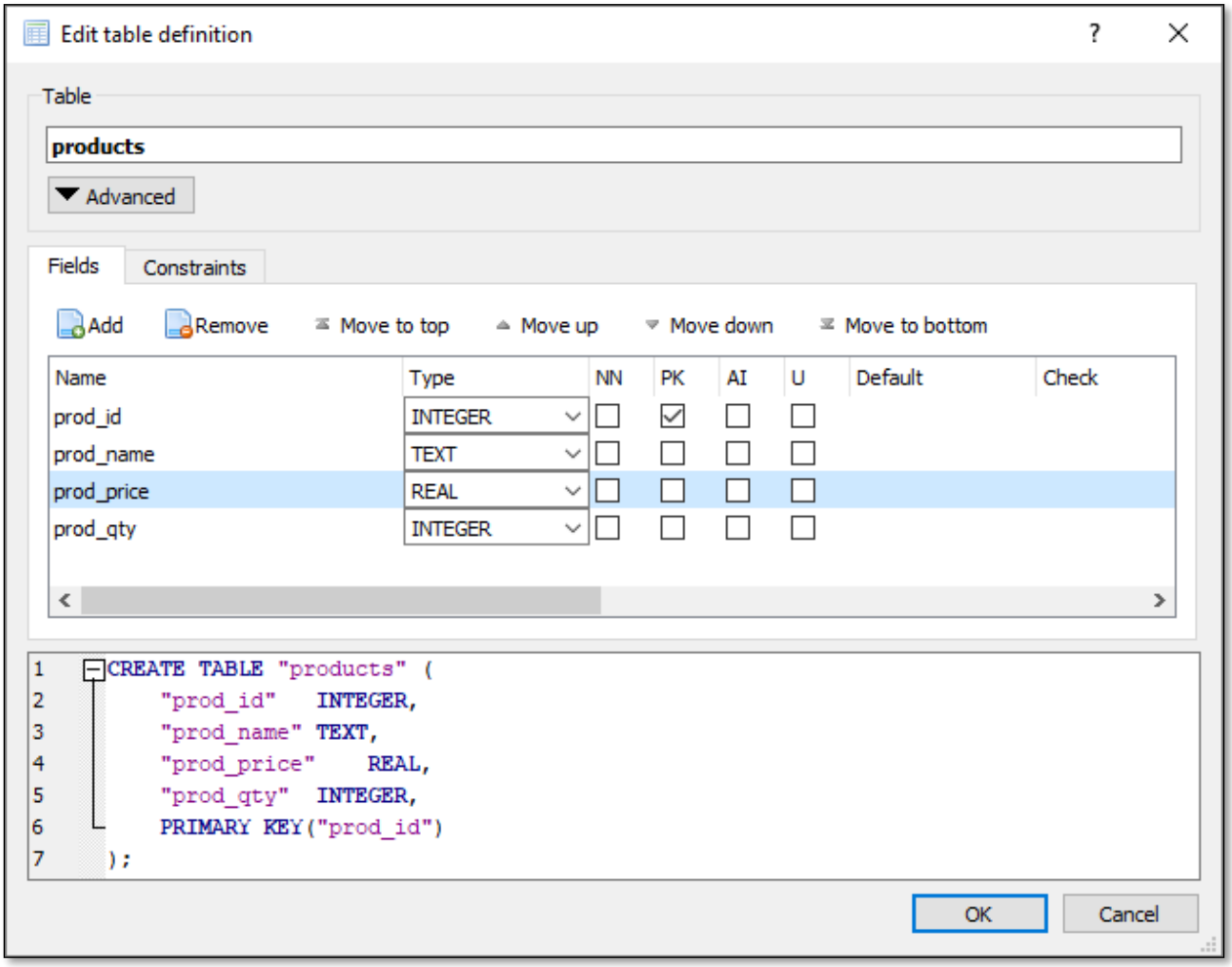
This is a handy tool to look at, troubleshoot, and manipulate your database. We will use it to learn the basics of SQL.

1. Go to https://sqlitebrowser.org
2. Go to the **Download** tab.
3. Download the **Windows PortableApp → DB Browser for SQLite - PortableApp**
4. Double Click the installation file. Click **Next**.
5. Click **Install**. Click **Finish**.
6. You will find a new folder: **SQLiteDatabaseBrowserPortable**
7. This folder can be moved anywhere, the program will work just fine.
8. In the folder you will find **SQLiteDatabaseBrowserPortable.exe**
9. Double Click the file. Click **OK** on the warning if there is one.

Create Database

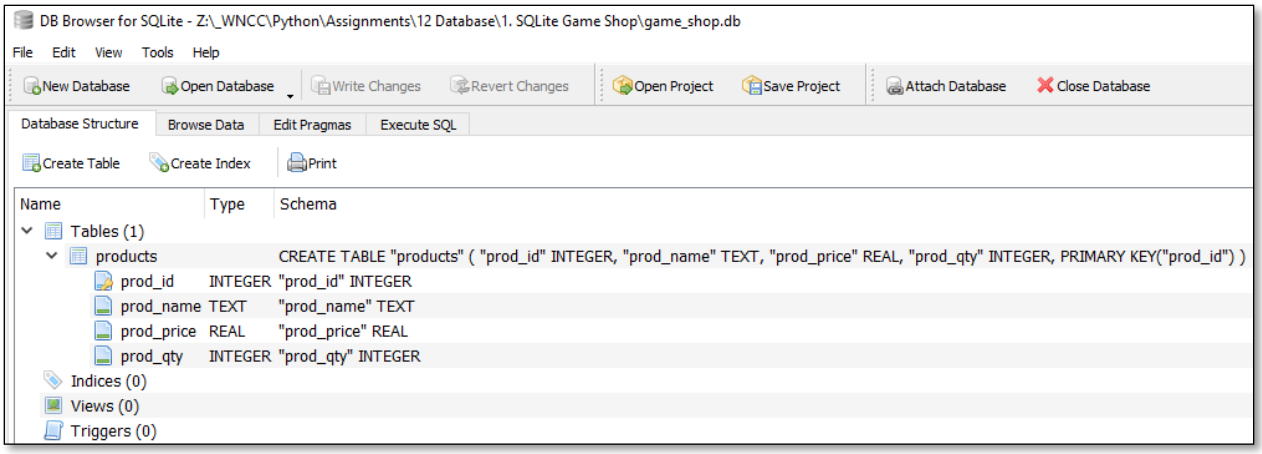
1. Double Click **SQLiteDatabaseBrowserPortable.exe**
2. Click **New Database.**
3. Filename: **game_shop.db →** Click **Save.**
4. Edit table definition
    a. Table: **products**
5. Fields:
    a. Click **Add**: Name: **prod_id**
    b. Click **Add**: Name: **prod_name**
    c. Click **Add**: Name: **prod_price**
    d. Click **Add:** Name: **prod_qty**

6. Make the setting changes as shown below. Notice the SQL code is created automatically. This is same code you would use to create this database and table in Python. SQL is SQL wherever it is used.



7. Click **OK** to create your table.

You should see the following result.



8. The changes you made are only in memory. To commit the changes to the database file, Click **Write Changes**.
9. Click **Close Database** to disconnect from the database.
10. Close DB Browser for SQLite.

Tutorial 2: Create a Database with Python

Explanation: How to Connect to an SQLite Database in Python

When we connect to an SQLite database, we are accessing data that resides in a file on our local computer.

1. To use SQLite, import **sqlite3**.

```
import sqlite3
```

2. Create a connection object using the **connect()** method and pass the name of the database file you want to access. If there is a file with that name, it will open that file. Otherwise, Python will create a file with the given name.
   The connection object represents the database in memory.

```
connection = sqlite3.connect("music.db")
```

3. A cursor object is used to send SQL and other commands to the database file.

```
cursor = connection.cursor()
```

4. Always close the cursor and connection to the database when the operation is done.

```
cursor.close()
connection.close()
```

Please create the following Python program: **music_library.py**

Please keep copies of each tutorial as you go in case something goes wrong.

```python
"""
    Name: music_library.py
    Author: William Loring
    Created: 01/04/23
    Use SQLite with Python
"""
# Import sqlite3 database library
import sqlite3

# Connect to or create a database file
# Create a connection object
connection = sqlite3.connect("music.db")

# Create a cursor object to interact with the database
cursor = connection.cursor()
print(" -- Connected to the database --")

# Always close the cursor and connection
cursor.close()
connection.close()
print(" -- Connection closed --\n")
```

Example run:

```
-- Connected to the database --
-- Connection closed --
```

Tutorial 3: Create a Table

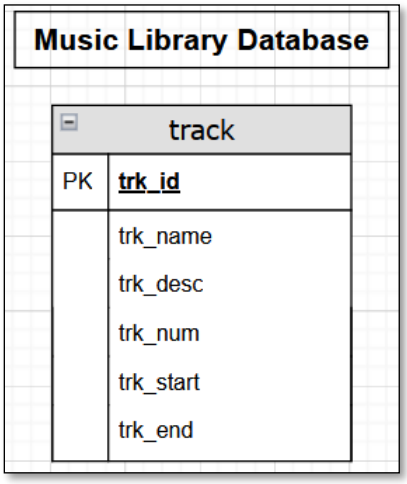
In an SQL database, data is stored in tables. Tables define a set of columns and fields, much like a spreadsheet. They contain 0 or more rows with data for each of the defined fields.

Let's create a table named **track** that stores the following data. This is a data dictionary.

Field Name	**Data Type**	**Description**
trk_id (primary key)	INTEGER	Unique track identifier
trk_name	TEXT	Track name
trk_desc	TEXT	Track description
trk_num	INTEGER	Track number
trk_start	REAL	Track start time

trk_end	REAL	Track end time

This is an ERD (Entity Relationship Diagram). This is a visual representation of our database fields.



We will create a **track** table using SQLite3 in Python.

Explanation

- The "**CREATE TABLE** “track” ..." multiline string is a SQL statement that creates a table named **products** with the columns described earlier:
    - **trk_id** of type INTEGER PRIMARY KEY
    - **trk_name** of type TEXT
    - **trk_desc** of type TEXT
    - **trk_num** of type INTEGER
    - **trk_start** of type REAL
    - **trk_end** of type REAL
- A **primary key** is a unique value that provides a unique identifier for each record. A primary key is typically a sequence of whole numbers (int).
- **connection.commit()** writes the data to the file.

SQL is a scripting language like Python. You can assign SQL code to a String variable using a multiline string.

```
SQL = """
CREATE TABLE IF NOT EXISTS track (
    trk_id        INTEGER PRIMARY KEY,
    trk_name      TEXT,
    trk_genre     TEXT,
    trk_num       INTEGER,
    trk_start     REAL,
    trk_end       REAL
);
"""
```

The result of the following code is the same as what we did earlier in DB Browser for SQLite.

The following code is inserted after line 16.

```
17 # --------------------- DROP TABLE --------------------------------------#
18 # SQL statements are text. SQL queries can be very long
19 # Create the SQL statement and assign it to a string
20 SQL = "DROP TABLE IF EXISTS track"
21 cursor.execute(SQL)
22
23 # --------------------- CREATE TABLE ------------------------------------#
24 SQL = """
25 CREATE TABLE IF NOT EXISTS track (
26     trk_id        INTEGER PRIMARY KEY,
27     trk_name      TEXT,
28     trk_genre     TEXT,
29     trk_num       INTEGER,
30     trk_start     REAL,
31     trk_end       REAL
32 );
33 """
34 cursor.execute(SQL)
35 print(" -- Table created in memory -- ")
```

Example run:

```
-- Connected to the database --
-- Table created in memory --
-- Table committed to disk --
-- Connection closed --
```

Tutorial 4: Insert Records

```
# --------------------- INSERT RECORDS ------------------------------------#
SQL = """
INSERT INTO track VALUES(
    0,
    'Stairway to Heaven',
    'Classic Rock',
    5,
    0.0,
    2.58
);
INSERT INTO track VALUES(
    1,
    'Old Time Rock and Roll',
    'Rock',
    2,
    0.0,
    5.21
);
INSERT INTO track VALUES(
    2,
    'Sister Christian',
    'Rock',
    2,
    0.0,
    2.56
);
INSERT INTO track VALUES(
    3,
    'Ramblin Man',
    'Southern Rock',
    5,
    0.0,
    2.25
);
"""
# executescript executes multiple SQL statements
cursor.executescript(SQL)
print(" -- Records inserted -- ")
```

Example run:

```
-- Connected to the database --
-- Table created in memory --
-- Records inserted --
-- Table committed to disk --
-- Connection closed --
```

Tutorial 5: Select Records

```
# -------------------SELECT ALL RECORDS ----------------------------------#
SQL = "SELECT * FROM track"
# fetchall() fetches the records returned by the SQL
# statement as a list of tuples
records = cursor.execute(SQL).fetchall()
for record in records:
    print(record)
```

Example run:

```
 -- Connected to the database --
 -- Table created in memory --
 -- Records inserted --
(0, 'Stairway to Heaven', 'Classic Rock', 5, 0.0, 2.58)
(1, 'Old Time Rock and Roll', 'Rock', 2, 0.0, 5.21)
(2, 'Sister Christian', 'Rock', 2, 0.0, 2.56)
(3, 'Ramblin Man', 'Southern Rock', 5, 0.0, 2.25)
 -- Table committed to disk --
 -- Connection closed --
```

Tutorial 6: Update Records

```
# ------------------- UPDATE RECORD -------------------------------------#
SQL = """
    UPDATE track
    SET trk_name = 'Bad Moon Rising'
    WHERE trk_id = 3
    """
cursor.execute(SQL)
print(" -- Record updated -- ")

# ------------------ SELECT ALL RECORDS ----------------------------------#
SQL = "SELECT * FROM track"
# fetchall() fetches the records returned by the SQL
# statement as a list of tuples
records = cursor.execute(SQL).fetchall()
for record in records:
    print(record)
```

Example run:

```
 -- Connected to the database --
 -- Table created in memory --
 -- Records inserted --
(0, 'Stairway to Heaven', 'Classic Rock', 5, 0.0, 2.58)
(1, 'Old Time Rock and Roll', 'Rock', 2, 0.0, 5.21)
(2, 'Sister Christian', 'Rock', 2, 0.0, 2.56)
(3, 'Ramblin Man', 'Southern Rock', 5, 0.0, 2.25)
 -- Record updated --
(0, 'Stairway to Heaven', 'Classic Rock', 5, 0.0, 2.58)
(1, 'Old Time Rock and Roll', 'Rock', 2, 0.0, 5.21)
(2, 'Sister Christian', 'Rock', 2, 0.0, 2.56)
(3, 'Bad Moon Rising', 'Southern Rock', 5, 0.0, 2.25)
 -- Table committed to disk --
 -- Connection closed --
```

Tutorial 7: Delete Records

Insert after Update Record.

```
# ------------------- DELETE RECORD ---------------------------------------#
SQL = "DELETE FROM track WHERE trk_id = 2"
cursor.execute(SQL)
print(" -- Record deleted -- ")
```

Example run:

```
 -- Connected to the database --
 -- Table created in memory --
 -- Records inserted --
(0, 'Stairway to Heaven', 'Classic Rock', 5, 0.0, 2.58)
(1, 'Old Time Rock and Roll', 'Rock', 2, 0.0, 5.21)
(2, 'Sister Christian', 'Rock', 2, 0.0, 2.56)
(3, 'Ramblin Man', 'Southern Rock', 5, 0.0, 2.25)
 -- Record updated --
 -- Record deleted --
(0, 'Stairway to Heaven', 'Classic Rock', 5, 0.0, 2.58)
(1, 'Old Time Rock and Roll', 'Rock', 2, 0.0, 5.21)
(3, 'Bad Moon Rising', 'Southern Rock', 5, 0.0, 2.25)
 -- Table committed to disk --
 -- Connection closed --
```

Assignment Submission

1. Attach the pseudocode or create a TODO.
2. Attach all tutorials and assignments.
3. Attach screenshots showing the successful operation of each tutorial program.

4. Submit in Blackboard.